Republic of the Philippines
BATANGAS STATE UNIVERSITY
The National Engineering University
Alangilan Campus
Golden Country Homes, Alangilan Batangas City, Batangas, Philippines 4200
Tel Nos.: (+63 43) 425-0139 local 2222 / 2223
E-mail Address: cics.alangilan@g.batstate-u.edu.ph | Website Address: http://www.batstate-u.edu.ph

College of Informatics and Computing Sciences

PROJECT PROPOSAL
OOP 101 – OBJECT-ORIENTED PROGRAMMING
Computer Science Department
1st Semester, AY 2026-2027

I. TEAM INFORMATION

Team Name:	**Sir Jayson Pogi Team**		
Team Number:		**Section:**	IT 2104

TEAM MEMBERS			
No.	**SR-Code**	**Name**	**GitHub Username**
1	25-05021	Cepillo, Steven Nicholas A.	Cepillo-25-05021
2	25-05052	Evangelista, Haira Joice A.	25-0505-cmd
3	25-07926	Macaraig, Austine Jay U.	ostin-dotcom
4	25-06457	Mendoza, Mark Airon O.	mendoza-25-06457
5	25-04787	Sanchez, Althea P.	sanchez417

II. PROJECT INFORMATION

Project Title:
RentLog: Java-Based Apartment Rental & Tenant Management System

Project Background
Managing apartment rooms and tenant records manually can lead to problems such as misplaced information, incorrect balance calculations, and unorganized payment and deposit records. These problems affect apartment owners, landlords, and property managers who are responsible for handling tenants and rental payments. To address these issues, RentLog is developed as a Java-based system that helps organize tenant information, monitor vacant and occupied rooms, record rental payments, calculate remaining balances, and track security deposits. It also provides monthly summary reports, making apartment management easier, faster, and more organized while reducing errors in manual record-keeping.

Project Objectives
General Objective
To develop a Java-based application that streamlines apartment management by tracking tenant information, rental contracts, payment histories, and security deposits using Object-Oriented Programming principles.

Specific Objectives
The project aims to:

- Implement the four core OOP principles (Abstraction, Encapsulation, Inheritance, and Polymorphism) across 5 core classes (Tenant, Apartment, Rental, Payment, and Deposit).
- Provide CRUD functionality (Add, View, Update, Delete) for tenant profiles and room statuses.
- Automatically calculate running balances based on monthly rent, payments made, and payment status (Paid, Partial, Unpaid).
- Track security deposits and advance payments along with their current status.
- Store payment records and generate monthly financial and occupancy reports using Java collections and file I/O.
- Provide secure dual-portal authentication (Landlord/Admin and Tenant sign-in) with role-based access control.

- Implement persistent data storage using the Google Gson library to automatically save and load system state across application sessions.

Target Users

Target User	How They Will Use the System
Landlord / Property Manager	Securely logs into the admin portal using administrator credentials, manages apartment room listings, adds/updates/deletes tenant records, records rental payments, tracks security deposits, and generates monthly summary reports.
Tenant (View Mode)	Securely logs into the individual tenant portal using personal credentials to view personal profile details, assigned rooms, monthly rent prices, remaining balances, and historical payment receipts.

Proposed Features

Feature	Description
Add Tenant	Registers a new tenant by entering their personal and contact information, assigned apartment room, and move-in date.
View Tenant Records	Shows all currently active tenant records, including their assigned room numbers and rental agreement details.
Update Tenant Information	Allows users to update existing tenant information, contact numbers, and assigned apartment rooms.
Delete Tenant	Allows users to remove a tenant's record when they move out and automatically changes the assigned room status to vacant.
Record Rental Payment	Allows users to record a payment by entering the amount, payment method, and date. Each payment is also given its own unique ID for easy tracking.
Calculate Remaining Balance	Automatically computes the balance due and updates status (Paid, Partial, Unpaid).
Dual-Portal Sign-In Authentication	Provides secure login screens for both administrators and tenants with credential validation and failed attempt limits.
Security Deposit & Advance Tracking	Tracks security deposits and advance payments, maintaining their holding status (Held, Applied, Refunded).
Monthly Financial & Occupancy Reporting	Aggregates total revenue collected, lists room occupancy statuses (Occupied vs. Vacant), and calculates total registered tenants.
JSON Data Persistence (Gson)	Automatically serializes and saves all system data (admin.json, tenants.json, apartments.json, payments.json, deposits.json) for seamless recovery upon restart.

III. JAVA AND OOP CONCEPTS

OOP Concept	How It Will Be Used
Classes and Objects	Instantiating core objects across primary and supporting classes: Tenant, Apartment, Rental, Payment, Deposit, Admin, and User.
Methods	Encapsulating specific behaviors and business logic, such as calculateBalance(), markOccupied(), markVacant(), updateContact(), getPaymentReceipt(), saveData(), and loadData().
Encapsulation	Keeping entity attributes private or protected (e.g., private double remainingBalance, protected String fullName) and exposing them securely via public getters, setters, and controlled mutation methods.
Inheritance	Extending the abstract base class User to derive specialized child roles: Admin (landlord management) and Tenant (portal view mode).
Polymorphism	Overriding abstract methods (e.g., getRoleDetails()) dynamically across different subclasses (Admin and Tenant) to provide specific role behaviors.
Abstraction	Utilizing abstract classes (User) and interfaces (Reportable) to enforce structural contracts for user roles and report summaries across the system.
Exception Handling	Managing runtime errors gracefully using try-catch blocks for menu input mismatches (NumberFormatException), invalid numeric entries, and file I/O errors (IOException).
Arrays/Data Handling	Utilizing dynamic Java collections (ArrayList<Tenant>, ArrayList<Apartment>, ArrayList<Payment>, etc.) alongside the Google Gson library for local JSON data persistence (tenants.json, apartments.json, payments.json, deposits.json, admin.json).

IV. PROPOSED CLASSES

Class	Purpose	Main Attributes	Main Methods
Tenant	Represents a tenant registered in the system (inherits from User and implements Reportable).	userID, fullName, contactNumber, roomNumber, moveInDate, username, password	getTenantInfo(), updateContact(), getRoleDetails(), getReportSummary()
Apartment	Represents an individual room/unit, pricing, and occupancy status.	roomNumber, monthlyRentalPrice, roomStatus (Occupied / Vacant)	markOccupied(), markVacant(), getRoomDetails()
Rental	Manages lease contract terms and agreement details.	monthlyRent, rentalDueDate, leaseExpirationDate	getRentalSummary()
Reportable (Interface)	Enforces abstraction and structure for generating summary reports.	None (Interface)	getReportSummary()
Payment	Handles payment transactions, tracking IDs, and running balance	paymentID, tenantID, amountPaid, paymentMethod, remainingBalance,	calculateBalance(), getPaymentReceipt()

	computations.	paymentStatus, paymentDate	
Deposit	Manages security deposits, advance payments, and their statuses.	tenantID, securityDeposit, advancePayment, depositStatus	applyDeposit(), refundDeposit(), getDepositSummary()
User (Abstract)	Serves as the base class demonstrating inheritance for all user roles.	userID, fullName, contactNumber	getRoleDetails() (abstract)
Admin	Manages administrator credentials and administrative access (inherits from User).	username, password	getRoleDetails()

V. PROGRAM FLOW

Start
↓
System Initialization & Data Loading (Gson reads local JSON files)
↓
Portal Selection (Landlord / Admin Portal vs. Tenant Portal)
↓
Authentication Screen (Enter Username and Password / Validate Credentials)
↓
Main Portal Menu (Display available options based on role)
↓
Select Operation (Choose CRUD action, view records, record payment, or view report)
↓
Enter Information (Provide required IDs, names, room numbers, or amounts)
↓
Validate Input & Error Handling (Check for valid numbers, existing rooms, or login attempts)
↓
Process Request (Execute business logic, update balances, mark room statuses, or serialize data via Gson)
↓
Display Result (Show success messages, receipts, summary reports, or profile details) ↓ Return to Portal Menu / Logout (Loop back or return to portal selection)
↓
Exit System

VI. TEAM RESPONSIBILITIES

Member	**Assigned Tasks**
Sanchez, Althea P.	Lead Programmer – Coordinates the implementation of Java classes, methods, and OOP features.
Cepillo, Steven Nicholas A.	Documentation – Maintains the README, testing documentation, screenshots. and assists in checking the application's functionality:
Evangelista, Haira Evangelista A.	System/Design Lead – Develops system requirements, class designs, UML diagrams, and overall application structure.
Macaraig, Austine Jay U.	Project Leader – Coordinates tasks, monitors deadlines, communicates with the instructor, and facilitates team meetings

Mendoza, Mark Airon O.	Documentation – Maintains the README, testing documentation, screenshots. and assists in checking the application's functionalit

VII. DEVELOPMENT SCHEDULE

Week	Activity	Expected Output
Week 1	Planning and requirements	Project proposal
Week 2	System and class design	Class/UML diagram
Week 3	Initial coding	Basic classes
Week 4–5	Feature development	Working features
Week 6	OOP implementation	Completed OOP features
Week 7	Testing and debugging	Tested application
Week 8	Documentation and presentation	Final project

VIII. EXPECTED OUTPUT

The team expects to produce a functional Java application that will streamline apartment rental operations and tenant management through a secure, dual-portal architecture. The application automates tenant profile tracking, room occupancy monitoring, rental payment calculations, security deposit tracking, and monthly financial reporting. Furthermore, it ensures reliable local data persistence using JSON file handling via the Gson library, while rigorously demonstrating core object-oriented programming (OOP) principles—including encapsulation, inheritance, polymorphism, and abstraction—across a well-structured multi-class architecture.

IX. PROJECT LIMITATIONS

- **Graphical User Interface (GUI):** Operates solely as a command-line interface (CLI) rather than a graphical desktop or web interface.
- **Database Management System:** Relies on local JSON files instead of a hosted relational database (e.g., MySQL).
- **Online Payment Gateway:** Payments must be manually recorded by the administrator rather than processed through live online payment channels.

X. GITHUB EXPOSITORY

Repository Name: RentLog
Link to Repository: https://github.com/sanchez417/RentLog

XI. APPROVAL

Prepared by:

Name	Signature

Republic of the Philippines
BATANGAS STATE UNIVERSITY
The National Engineering University
Alangilan Campus
Golden Country Homes, Alangilan Batangas City, Batangas, Philippines 4200
Tel Nos.: (+63 43) 425-0139 local 2222 / 2223
E-mail Address: cics.alangilan@g.batstate-u.edu.ph | Website Address: http://www.batstate-u.edu.ph

College of Informatics and Computing Sciences

Instructor's Comments:

Project Status:

□ Approved
□ Approved with Revisions
□ Revise and Resubmit

Name of Instructor:

Ms. FATIMA MARIE P. AGDON, MSCS

Date Signed: _____